

PORTADA

Técnicas de Programación

Unidad 2 — Paradigma Objetual: Clases Abstractas y Static

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

**Hay clases que existen a propósito
para que nunca crees un objeto a partir de ellas.**



TRANSICIÓN

De la herencia a la herencia incompleta a propósito

En la sesión de herencia construiste jerarquías donde la superclase (`Persona`) sí tenía sentido por sí sola. Pero, ¿qué significaría crear "una figura geométrica" sin decir si es un círculo o un cuadrado? No tiene sentido — y Java tiene una forma de **prohibirlo explícitamente**.

CONCEPTO

Clase abstracta

Una **clase abstracta** se declara con la palabra clave `abstract` y representa un concepto **incompleto a propósito**: sirve como base común para sus subclases, pero **no se puede instanciar directamente** con `new`.

```
public abstract class Figura {  
    // algunos métodos pueden tener cuerpo (concretos)  
    // otros pueden no tenerlo (abstractos) – ver siguiente  
}
```

Se usa cuando varias clases comparten estructura y parte de comportamiento, pero cada una debe completar el resto **a su manera**.

CONCEPTO

Métodos abstractos

Un **método abstracto** se declara solo con su firma, terminada en punto y coma, **sin cuerpo** (sin llaves `{}`): declara *qué* debe poder hacer cada subclase, sin decir *cómo*. Solo puede existir dentro de una clase también declarada `abstract`.

```
public abstract class Figura {  
    public abstract double area(); // sin cuerpo: cada subclase decide cómo calcularla  
  
    public String etiqueta() {      // método concreto: sí tiene cuerpo  
        return "Esta figura tiene un área calculable.";  
    }  
}
```

CONTRAEJEMPLO

Lo que Java rechaza: instanciar directamente

```
Figura f = new Figura(); // Error de compilación:  
                        // Figura is abstract; cannot be instantiated
```

El compilador impide crear un objeto de una clase abstracta **porque le faltaría implementación** (el método `area()` no tiene cuerpo) — no habría forma de ejecutar ese método si alguien lo llamara.

CONCEPTO

La subclase concreta completa el contrato

Una subclase que **no** se declara `abstract` (una clase **concreta**) está obligada a implementar **todos** los métodos abstractos heredados — de lo contrario, no compila.

```
public class Circulo extends Figura {
    private double radio;
    public Circulo(double radio) { this.radio = radio; }

    @Override
    public double area() {
        return Math.PI * radio * radio;
    }
}

public class Cuadrado extends Figura {
    private double lado;
    public Cuadrado(double lado) { this.lado = lado; }

    @Override
    public double area() { return lado * lado; }
}
```

INTERACCIÓN

Diseñen el contrato

En parejas: dada una clase abstracta `Empleado` con el método abstracto `calcularSalario()` , definan en papel:

- Dos subclases concretas (ej. `EmpleadoDeNomina` y `EmpleadoPorContrato`) con una fórmula distinta cada una.
- Un método concreto que podría vivir en `Empleado` porque no cambia entre subclases (ej. `mostrarNombre()`).

6 minutos · justifiquen por qué cada método va abstracto o concreto

TRANSICIÓN

Datos que no pertenecen a ningún objeto en particular

Hasta ahora, cada atributo vive dentro de un objeto específico (`ana.nombre` es distinto de `luis.nombre`). Pero hay información que tiene sentido **compartir entre todos los objetos de una clase** — o que ni siquiera necesita un objeto para existir. Para eso está el modificador `static` .

CONCEPTO

El modificador `static`

Un miembro (atributo o método) marcado `static` **pertenece a la clase, no a ningún objeto en particular**. Existe una sola copia de un atributo `static`, compartida por todos los objetos de esa clase, sin importar cuántos se creen.



CONCEPTO

Atributo `static` : un contador compartido

```
public class Estudiante {  
    private String nombre;  
    private static int totalCreados = 0;    // una sola copia, de la clase  
  
    public Estudiante(String nombre) {  
        this.nombre = nombre;  
        totalCreados++;                    // se incrementa cada vez que se crea uno  
    }  
}
```

```
new Estudiante("Ana");  
new Estudiante("Luis");  
System.out.println(Estudiante.totalCreados); // 2 – se accede con el nombre de la clase
```

Se accede con el nombre de la clase (`Estudiante.totalCreados`), no con un objeto — aunque Java también lo permita, no es la forma correcta de leerlo.

CONCEPTO

Método `static` : no necesita un objeto

Un método `static` se invoca directamente con el nombre de la clase, sin crear ningún objeto primero. Ya los usas: `Math.sqrt(16)` o `Integer.parseInt("42")` son métodos estáticos de clases de la librería estándar de Java.

```
public class Calculadora {  
    public static int sumar(int a, int b) {  
        return a + b;  
    }  
}  
  
int resultado = Calculadora.sumar(2, 3); // sin "new Calculadora()"
```

CONTRA EJEMPLO

Lo que un método `static` no puede hacer

```
public class Estudiante {  
    private String nombre;    // atributo de instancia  
  
    public static void mostrarNombre() {  
        System.out.println(nombre); // Error de compilación:  
                                    // non-static field cannot be referenced  
                                    // from a static context  
    }  
}
```

Un método `static` se ejecuta **sin ningún objeto asociado**, así que no tiene forma de saber a cuál `nombre` se refiere — por eso no puede acceder a atributos de instancia (ni usar `this`) directamente.

CONCEPTO

Constantes de clase: `static final`

Combinar `static` (una sola copia, de la clase) con `final` (no se puede reasignar después de inicializarse) define una **constante de clase**. Por convención, su nombre va en mayúsculas separadas por guion bajo.

```
public class Impuestos {  
    public static final double IVA = 0.19;  
}  
  
double total = precio + (precio * Impuestos.IVA);
```

INTERACCIÓN

¿Static o de instancia?

Para cada elemento, indiquen si debería ser `static` o de instancia, y por qué:

1. El nombre de un `Estudiante` .
2. El número máximo de créditos permitidos por semestre (igual para todos los estudiantes).
3. Un método `convertirPulgadasACm(double pulgadas)` que no depende de ningún objeto.
4. El promedio acumulado de un `Estudiante` específico.

5 minutos · respondan en el chat

SÍNTESIS

Lo que hay que llevarse de hoy

1. Una clase abstracta (`abstract`) no se puede instanciar; existe para ser extendida.
2. Un método abstracto no tiene cuerpo — toda subclase concreta está obligada a implementarlo.
3. `static` marca un miembro como perteneciente a la clase, no a un objeto: existe una sola copia compartida.
4. Un método `static` no puede acceder a atributos de instancia ni usar `this` , porque no está asociado a ningún objeto.



CIERRE

Antes de la próxima sesión

- Escribir en el IDE la clase abstracta `Figura` con `Circulo` y `Cuadrado`, y comprobar que `new Figura()` efectivamente no compila.
- Pensar qué pasaría si dos clases sin relación de herencia (ej. `Ave` y `Avion`) necesitaran compartir el comportamiento "volar" — no pueden heredar de una misma superclase si no tienen nada más en común.

La próxima sesión resuelve justo ese caso con interfaces, y arranca el tema de polimorfismo.

